

compress_logs.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Logger - Compress Logs
4 # =====
5 # Description:
6 #   Script for log file compression - combines repeated lines.
7 #
8 # Usage Examples:
9 #   CLI:
10 #       python -m logger.compress_logs
11 #   Python API:
12 #       from logger.compress_logs import compress_lines
13 #
14 #       res = compress_lines()
15 #
16 # File: compress_logs.py
17 # Project: ai-breadboard
18 # Package: logger
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-01 13:29:02
22 # =====
23
24 """Script for log file compression - combines repeated lines.
25
26 Reduces log file size by consolidating identical lines into [N
27 x] format.
28
29 Usage: python scripts/compress_logs.py"""
30
31 import sys
32 from collections import Counter
33 from pathlib import Path
34 from typing import List, Tuple
```

```

33
34 def compress_lines(lines: List[str], min_repeat: int=2) -> Lis
    t[str]:
35     """Compress repeated lines into [Nx] text format.
36
37     Args:
38         lines (List[str]): Input lines to compress.
39         min_repeat (int): Minimum repetitions to trigger compr
        ession.
40
41     Returns:
42         List[str]: Compressed lines.
43     """
44     counter = Counter(lines)
45     result = []
46     for line, count in counter.items():
47         stripped = line.strip()
48         if not stripped:
49             continue
50         if count >= min_repeat:
51             result.append(f'[{count}x] {stripped}')
52         else:
53             result.append(stripped)
54     return result
55
56 def compress_log_file(input_path: Path, output_path: Path=None
    , min_repeat: int=2) -> Tuple[int, int]:
57     """Compress log file.
58
59     Args:
60         input_path (Path): Input log file path.
61         output_path (Path): Output log file path (default: .co
        mpressed.log).
62         min_repeat (int): Minimum repetitions to compress.
63
64     Returns:
65         Tuple[int, int]: (original_lines_count, compressed_lin
        es_count)

```

```

66 |     """
67 |     with open(input_path, 'r', encoding='utf-8', errors='ignor
    | e') as f:
68 |         lines = f.readlines()
69 |         if not lines:
70 |             return (0, 0)
71 |         original_count = len(lines)
72 |         compressed = compress_lines(lines, min_repeat)
73 |         if output_path is None:
74 |             output_path = input_path.with_suffix('.compressed.log'
    | )
75 |         with open(output_path, 'w', encoding='utf-8') as f:
76 |             f.write('\n'.join(compressed))
77 |             if compressed:
78 |                 f.write('\n')
79 |         return (original_count, len(compressed))
80 |
81 | def main():
82 |     """Main entry point for log compression."""
83 |     from logger import logger
84 |     logs_dir = Path(__file__).resolve().parent.parent.parent /
    | 'tmp' / 'logs'
85 |     if not logs_dir.exists():
86 |         logger.error(f'Logs directory not found: {logs_dir}')
87 |         return
88 |     log_files = list(logs_dir.glob('*.log'))
89 |     if not log_files:
90 |         logger.info('No log files found')
91 |         return
92 |     logger.info(f'Found files to compress: {len(log_files)}')
93 |     for log_file in log_files:
94 |         original, compressed = compress_log_file(log_file)
95 |         if original > 0:
96 |             compression_pct = 100 - compressed * 100 // origin
    | al
97 |             logger.info(f'{log_file.name}: {original} → {compr
    | essed} lines (compression {compression_pct}%)')
98 |         else:

```

```
99 |         logger.info(f'{log_file.name}: empty file')
100 | if __name__ == '__main__':
101 |     main()
```

conftest.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Logger - Conftest
4 # =====
5 # Description:
6 #   pytest configuration for logger module testing.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from logger.conftest import project_root
11 #
12 #       res = project_root()
13 #
14 # File: conftest.py
15 # Project: ai-breadboard
16 # Package: logger
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:29:02
20 # =====
21
22 """pytest configuration for logger module testing.
23
24 Provides fixtures and configuration for unit tests of src.logg
25 er module."""
26
27 import pytest
28 import sys
29 from pathlib import Path
30 from header import __root__
31 sys.path.insert(0, str(__root__))
32
33 @pytest.fixture(scope='session')
```

```
33 def project_root():
34     """Fixture to get project root directory."""
35     return __root__
36
37 @pytest.fixture(scope='session')
38 def logger_module_path():
39     """Фикстура для получения пути к модулю логирования."""
40     return __root__ / 'src' / 'logger'
41
42 @pytest.fixture
43 def temp_log_dir(tmp_path):
44     """Фикстура для создания временной директории логов."""
45     log_dir = tmp_path / 'logs'
46     log_dir.mkdir()
47     return log_dir
48
49 @pytest.fixture
50 def temp_reports_dir(tmp_path):
51     """Фикстура для создания временной директории отчётов."""
52     reports_dir = tmp_path / 'reports'
53     reports_dir.mkdir()
54     return reports_dir
55
56 @pytest.fixture
57 def mock_logger_config():
58     """Фикстура для мокирования конфигурации логгера."""
59     from unittest.mock import MagicMock
60     config = MagicMock()
61     config.mode = 'dev'
62     config.debug = True
63     return config
64
65 @pytest.fixture
66 def mock_ai_model():
67     """Фикстура для мокирования AI модели."""
68     from unittest.mock import AsyncMock
69     model = AsyncMock()
70     model.ask.return_value = '# Analysis Report\n\nNo issues f
```

```

    ound.'
71     return model
72
73 def pytest_configure(config):
74     """Регистрирует пользовательские markers для pytest."""
75     config.addinvalue_line('markers', 'slow: marks tests as s
low (deselect with \'-m "not slow"\')')
76     config.addinvalue_line('markers', 'integration: marks tes
ts as integration tests')
77     config.addinvalue_line('markers', 'unit: marks tests as u
nit tests')
78     config.addinvalue_line('markers', 'performance: marks tes
ts for performance testing')
79
80 def pytest_collection_modifyitems(config, items):
81     """Модифицирует собранные тесты."""
82     for item in items:
83         if 'test_' in item.name:
84             if not any((marker.name == 'unit' for marker in it
em.iter_markers())):
85                 item.add_marker(pytest.mark.unit)
86 pytest_plugins = ['pytest_asyncio']

```

header.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Logger - Header
4 # =====
5 # Description:
6 #   Module for project root path determination and meta information initialization.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       import logger.header as header
11 #
12 # File: header.py
13 # Project: ai-breadboard
14 # Package: logger
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-01 13:29:02
18 # =====
19
20 """Module for project root path determination and meta information initialization.
21
22 Solves two primary tasks:
23 1. Determines project root directory by searching for marker files
24 2. Loads and initializes project meta information from configuration files"""
25
26 import sys
27 import json
28 import logging
29 from packaging.version import Version
```

```

30 from pathlib import Path
31 from typing import Dict, Optional, Tuple
32 from header import __root__, set_project_root
33
34 def _load_project_settings() -> Dict[str, str]:
35     """
36     Loads метайнформацию проекта из файла конфигурации.
37
38     Поиск конфигурации:
39     1. Сначала пытается найти settings.json в core/
40     2. Если не найден, Returns empty dictionary
41
42     Returns:
43         Dict: Dictionary с параметрами проекта или empty dicti
44         onary при ошибке.
45     """
46     settings: Dict[str, str] = {}
47     try:
48         from src import gs
49         if gs:
50             settings_path = gs.path.root / 'core' / 'settings.
51             json'
52             if settings_path.exists():
53                 with open(settings_path, 'r', encoding='utf-8'
54                 ) as settings_file:
55                     settings = json.load(settings_file)
56             except (ImportError, FileNotFoundError, json.JSONDecodeErr
57             or, AttributeError) as ex:
58                 logging.debug(f'Не удалось загрузить settings.json: {e
59                 x}')
60         return settings
61
62 def _load_project_documentation() -> str:
63     """
64     Loads документацию проекта из README файла.
65
66     Returns:
67         str: Содержимое документации или пустая string при оши

```

```

62 | бке.
63 |     """
64 |     doc_str: str = ''
65 |     try:
66 |         from src import gs
67 |         if gs:
68 |             readme_path = gs.path.root / 'core' / 'README.MD'
69 |             if readme_path.exists():
70 |                 with open(readme_path, 'r', encoding='utf-8')
71 | as readme_file:
72 |                     doc_str = readme_file.read()
73 |     except (ImportError, FileNotFoundError, AttributeError) as
74 | ex:
75 |         logging.debug(f'Не удалось загрузить документацию: {ex
76 | }')
77 |     return doc_str
78 | settings: Dict[str, str] = _load_project_settings()
79 | doc_str: str = _load_project_documentation()
80 | __project_name__: str = settings.get('project_name', 'ai-bread
81 | board') if settings else 'ai-breadboard'
82 | 'Название проекта'
83 | __version__: str = settings.get('version', '1.0.0') if setting
84 | s else '1.0.0'
85 | 'Версия проекта'
86 | __doc__: str = doc_str if doc_str else 'AI Breadboard - Интелл
87 | ектуальная система анализа'
88 | 'Документация проекта'
89 | __author__: str = settings.get('author', 'Development Team') i
90 | f settings else 'Development Team'
91 | 'Автор проекта'
92 | __copyright__: str = settings.get('copyright', '© 2026 Develop
93 | ment Team') if settings else '© 2026 Development Team'
94 | 'Копирайт проекта'
95 | __cofee__: str = settings.get('cofee', 'Treat the developer to
96 | a cup of coffee for boosting enthusiasm in development: https
97 | ://boosty.to/hypo69') if settings else 'Treat the developer to
98 | a cup of coffee for boosting enthusiasm in development: https
99 | ://boosty.to/hypo69'

```


logger.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Logger - Logger
4 # =====
5 # Description:
6 #   Модуль реализации компонента `SingletonMeta`.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from logger.logger import SingletonMeta
11 #
12 #       service = SingletonMeta()
13 #
14 # File: logger.py
15 # Project: ai-breadboard
16 # Package: logger
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 18:19:00
20 # =====
21
22 from __future__ import annotations
23 """Модуль реализации компонента `SingletonMeta`."""
24
25 import logging
26 import logging.handlers
27 import colorama
28 import datetime
29 import json
30 import inspect
31 import sys
32 import threading
33 import queue
```

```

34 import atexit
35 import tempfile
36 from pathlib import Path
37 from typing import Optional, Tuple, Dict, Any
38 from src.utils.printer import pformat
39 from types import SimpleNamespace
40 from collections import Counter
41 import header
42 from header import __root__
43 colorama.init(autoreset=False)
44 TEXT_COLORS: Dict[str, str] = {'black': colorama.Fore.BLACK, '
    red': colorama.Fore.RED, 'green': colorama.Fore.GREEN, 'yellow
    ': colorama.Fore.YELLOW, 'blue': colorama.Fore.BLUE, 'magenta'
    ': colorama.Fore.MAGENTA, 'cyan': colorama.Fore.CYAN, 'white':
    colorama.Fore.WHITE, 'light_gray': colorama.Fore.LIGHTBLACK_EX
    , 'light_red': colorama.Fore.LIGHTRED_EX, 'light_green': color
    ama.Fore.LIGHTGREEN_EX, 'light_yellow': colorama.Fore.LIGHTYEL
    LOW_EX, 'light_blue': colorama.Fore.LIGHTBLUE_EX, 'light_magen
    ta': colorama.Fore.LIGHTMAGENTA_EX, 'light_cyan': colorama.For
    e.LIGHTCYAN_EX}
45 BG_COLORS: Dict[str, str] = {'black': colorama.Back.BLACK, 're
    d': colorama.Back.RED, 'green': colorama.Back.GREEN, 'yellow':
    colorama.Back.YELLOW, 'blue': colorama.Back.BLUE, 'magenta':
    colorama.Back.MAGENTA, 'cyan': colorama.Back.CYAN, 'white': co
    lorama.Back.WHITE, 'light_gray': colorama.Back.LIGHTBLACK_EX,
    'light_red': colorama.Back.LIGHTRED_EX, 'light_green': coloram
    a.Back.LIGHTGREEN_EX, 'light_yellow': colorama.Back.LIGHTYELLO
    W_EX, 'light_blue': colorama.Back.LIGHTBLUE_EX, 'light_magenta
    ': colorama.Back.LIGHTMAGENTA_EX, 'light_cyan': colorama.Back.
    LIGHTCYAN_EX}
46
47 class SingletonMeta(type):
48     """
49     Метакласс для реализации паттерна Singleton.
50     Гарантирует, что class имеет только один экземпляр и предо
    ставляет
51     глобальную точку доступа к этому экземпляру.
52

```

```

53 |     Потокобезопасен благодаря использованию Lock.
54 |     """
55 |     _instances: Dict = {}
56 |     _lock: threading.Lock = threading.Lock()
57 |
58 |     def __call__(cls, *args, **kwargs):
59 |         """
60 |         Creates или Returns существующий экземпляр класса.
61 |
62 |         Returns:
63 |             Единственный экземпляр класса.
64 |         """
65 |         if cls not in cls._instances:
66 |             with cls._lock:
67 |                 if cls not in cls._instances:
68 |                     instance = super().__call__(*args, **kwargs
69 | s)
70 |                     cls._instances[cls] = instance
71 |         return cls._instances[cls]
72 |
73 | class JsonFormatter(logging.Formatter):
74 |     """
75 |     Кастомный форматтер для логирования в JSON формате.
76 |
77 |     Преобразует LogRecord в JSON строку с полями:
78 |     - timestamp: временная метка записи
79 |     - level: уровень логирования (INFO, DEBUG и т.д.)
80 |     - message: текст сообщения
81 |     - exc_info: Info об исключении (если присутствует)
82 |     """
83 |
84 |     def format(self, record: logging.LogRecord) -> str:
85 |         """
86 |         Форматирует лог-запись в JSON.
87 |
88 |         Args:
89 |             record: LogRecord для форматирования.

```

```

90 |         Returns:
91 |             JSON string с информацией о логe.
92 |             """
93 |         log_entry: Dict = {'timestamp': self.formatTime(record
| , self.datefmt), 'level': record.levelname, 'levelname': recor
| d.levelname, 'message': record.getMessage().replace(' ', ''),
| 'exc_info': self.formatException(record.exc_info) if record.e
| xc_info else None}
94 |         return json.dumps(log_entry, ensure_ascii=False)
95 |
96 | class CompressingHandler(logging.handlers.RotatingFileHandler)
| :
97 |     """
98 |     Обработчик файловых логов с компрессией повторяющихся запи
| сей и ротацией.
99 |
100 |     Особенности:
101 |     - Буферизирует записи и сжимает их в формате [Nx] message
102 |     - Поддерживает ротацию файлов по размеру
103 |     - Потокбезопасен при многопоточном доступе
104 |     - Гарантирует сохранение логов при падении процесса
105 |
106 |     Parameters:
107 |         filename: Путь к файлу лога
108 |         maxBytes: Максимальный размер файла перед ротацией (по
| умолчанию 10MB)
109 |         backupCount: Количество резервных файлов (по умолчанию
| 5)
110 |     """
111 |
112 |     def __init__(self, filename: str, maxBytes: int=10 * 1024
| * 1024, backupCount: int=5, encoding: str='utf-8'):
113 |         """
114 |         Инициализирует обработчик логов с ротацией.
115 |
116 |     Args:
117 |         filename: Путь к файлу лога.
118 |         maxBytes: Максимальный размер файла в байтах.

```

```

119 |         backupCount: Количество сохраняемых резервных файл
    | ов.
120 |         encoding: Кодировка файла.
121 |         """
122 |         super().__init__(filename, maxBytes=maxBytes, backupCo
    | unt=backupCount, encoding=encoding, delay=False)
123 |         self.buffer: Dict[str, int] = {}
124 |         self._lock = threading.Lock()
125 |         self._dirty = False
126 |
127 |     def emit(self, record: logging.LogRecord) -> None:
128 |         """
129 |         Добавляет запись в буфер с компрессией.
130 |
131 |         Args:
132 |             record: LogRecord для добавления.
133 |         """
134 |         try:
135 |             msg = self.format(record)
136 |             with self._lock:
137 |                 self.buffer[msg] = self.buffer.get(msg, 0) + 1
138 |                 self._dirty = True
139 |                 if len(self.buffer) > 100:
140 |                     self.flush()
141 |         except Exception:
142 |             self.handleError(record)
143 |
144 |     def flush(self) -> None:
145 |         """Сбрасывает буфер в файл с компрессией повторяющихся
    | записей."""
146 |         with self._lock:
147 |             if not self.buffer or not self._dirty:
148 |                 return
149 |             try:
150 |                 if not self.stream:
151 |                     self.stream = self._open()
152 |                 for msg, count in self.buffer.items():
153 |                     if count > 1:

```

```

154         self.stream.write(f'[{count}x] {msg}\n
    ')
155         else:
156             self.stream.write(f'{msg}\n')
157             self.stream.flush()
158             self.buffer.clear()
159             self._dirty = False
160     except Exception:
161         pass
162
163     def close(self) -> None:
164         """Закрывает обработчик, предварительно сбрасывая буфе
    р."""
165         self.flush()
166         super().close()
167
168     def compress_lines(lines: list, min_repeat: int=2) -> list:
169         """Сжимает повторяющиеся строки в формат [Nx] text."""
170         counter = Counter(lines)
171         result = []
172         for line, count in counter.items():
173             stripped = line.strip()
174             if not stripped:
175                 continue
176             if count >= min_repeat:
177                 result.append(f'[{count}x] {stripped}')
178             else:
179                 result.append(stripped)
180         return result
181
182     class PrettyConsoleFormatter(logging.Formatter):
183         """
184         Консольный форматтер с автоматическим форматированием JSON
    и вложенных структур.
185         """
186
187         def format(self, record: logging.LogRecord) -> str:
188             """

```

```

189 |         Форматирует лог-запись для консоли с применением pformat
    | at к JSON и структурам данных.
190 |
191 |     Args:
192 |         record: LogRecord для форматирования.
193 |
194 |     Returns:
195 |         Отформатированная строка лога.
196 |     """
197 |     from src.utils.printer import pformat
198 |     try:
199 |         orig_msg = record.getMessage()
200 |         formatted_msg = pformat(orig_msg)
201 |         orig_record_msg = record.msg
202 |         orig_args = record.args
203 |         record.msg = formatted_msg
204 |         record.args = None
205 |         result = super().format(record)
206 |         record.msg = orig_record_msg
207 |         record.args = orig_args
208 |         return result
209 |     except Exception:
210 |         return super().format(record)
211 |
212 | class Logger(metaclass=SingletonMeta):
213 |     """
214 |     Универсальный логгер с поддержкой цветного вывода, файловы
    | х логов и JSON формата.
215 |
216 |     Реализует паттерн Singleton и гарантирует единственный экз
    | емплар логгера во всем приложении.
217 |
218 |     Основные возможности:
219 |     - Цветной вывод в консоль с поддержкой цветов текста и фон
    | а
220 |     - Splitting логов по типам (info, debug, errors)
221 |     - JSON форматирование для структурированного анализа
222 |     - Ротация файлов логов по размеру

```

```
223 |     - Умная маршрутизация логов по модулям (FastAPI, Gemini, P
    | laywright и т.д.)
224 |     - Автоматическое disconnection DEBUG логов в production ре
    | жиме
225 |
226 |     Атрибуты:
227 |         log_files_path: Путь к директории с логами
228 |         info_log_path: Путь к файлу информационных логов
229 |         debug_log_path: Путь к файлу отладочных логов
230 |         errors_log_path: Путь к файлу ошибок
231 |         json_log_path: Путь к JSON логам
232 |         is_debug_mode: Флаг режима отладки
233 |     """
234 |
235 |     def __init__(self, info_log_path: Optional[str]=None, debu
    | g_log_path: Optional[str]=None, errors_log_path: Optional[str]
    | =None, json_log_path: Optional[str]=None):
236 |         """
237 |         Инициализирует логгер с заданными путями файлов логов.
238 |
239 |         Args:
240 |             info_log_path: Имя файла для INFO логов.
241 |             debug_log_path: Имя файла для DEBUG логов.
242 |             errors_log_path: Имя файла для ERROR логов.
243 |             json_log_path: Имя файла для JSON логов.
244 |         """
245 |         self.log_files_path: Path = Path(tempfile.gettempdir()
    | ) / 'ai-breadboard' / 'logs'
246 |         self.info_log_path: Path = self.log_files_path / (info
    | _log_path or 'info.log')
247 |         self.debug_log_path: Path = self.log_files_path / (deb
    | ug_log_path or 'debug.log')
248 |         self.errors_log_path: Path = self.log_files_path / (er
    | rors_log_path or 'errors.log')
249 |         self.json_log_path: Path = self.log_files_path / (json
    | _log_path or 'log.json')
250 |         self.fastapi_log_path: Path = self.log_files_path / 'f
    | astapi.log'
```

```

251         self.gemini_log_path: Path = self.log_files_path / 'ge
mini.log'
252         self.playwright_log_path: Path = self.log_files_path /
'playwright.log'
253         self.yt_dlp_log_path: Path = self.log_files_path / 'yt
_dlp.log'
254         self.log_files_path.mkdir(parents=True, exist_ok=True)
255         for log_path in [self.info_log_path, self.debug_log_pa
th, self.errors_log_path, self.json_log_path, self.fastapi_log
_path, self.gemini_log_path, self.playwright_log_path, self.yt
_dlp_log_path]:
256             log_path.touch(exist_ok=True)
257             if sys.platform == 'win32':
258                 for stream in (sys.stdout, sys.stderr, getattr(sys
, '__stdout__', None), getattr(sys, '__stderr__', None)):
259                     if stream is not None and hasattr(stream, 'rec
onfigure'):
260                         try:
261                             stream.reconfigure(encoding='utf-8', e
rrors='replace')
262                         except Exception:
263                             pass
264                 console_formatter = PrettyConsoleFormatter('%(asctime)
s - %(levelname)s - %(message)s')
265                 target_stream = sys.__stdout__ if getattr(sys, '__stdo
ut__', None) is not None else sys.__stderr__
266                 if target_stream is not None:
267                     console_handler = logging.StreamHandler(target_str
eam)
268                     console_handler.setFormatter(console_formatter)
269                 else:
270                     console_handler = logging.NullHandler()
271                 self.logger_console: logging.Logger = logging.getLogge
r('logger_console')
272                 self.logger_console.setLevel(logging.DEBUG)
273                 self.logger_console.propagate = False
274                 if not self.logger_console.handlers:
275                     self.logger_console.addHandler(console_handler)

```

```

276 |         root_logger = logging.getLogger()
277 |         if not root_logger.handlers:
278 |             root_logger.addHandler(console_handler)
279 |             root_logger.setLevel(logging.INFO)
280 |         else:
281 |             for h in root_logger.handlers:
282 |                 if isinstance(h, logging.StreamHandler):
283 |                     h.setFormatter(console_formatter)
284 |             for noisy_logger in ['httpx', 'httpcore', 'google', 'g
oogle.genai', 'urllib3']:
285 |                 logging.getLogger(noisy_logger).setLevel(logging.W
ARNING)
286 |         self._setup_debug_mode()
287 |         self._setup_file_handlers()
288 |         self._setup_module_loggers()
289 |         atexit.register(self._cleanup)
290 |
291 |     def _setup_debug_mode(self) -> None:
292 |         """Определяет, находится ли приложение в режиме отладк
и."""
293 |         try:
294 |             import os
295 |             from dotenv import load_dotenv
296 |             load_dotenv(__root__ / '.env')
297 |             try:
298 |                 from src.config import server_cfg, logging_cfg
299 |                 log_debug = getattr(logging_cfg, 'debug', None
)
300 |                 log_level = str(getattr(logging_cfg, 'level',
'')).lower()
301 |                 mode_val = getattr(server_cfg, 'mode', 'dev').
lower()
302 |                 server_debug = getattr(server_cfg, 'debug', Tr
ue)
303 |                 if log_debug is not None:
304 |                     self.is_debug_mode = bool(log_debug)
305 |                 elif log_level:
306 |                     self.is_debug_mode = log_level == 'debug'

```

```

307         else:
308             self.is_debug_mode = mode_val in ('dev', '
debug') or server_debug
309         except ImportError:
310             mode_val = os.getenv('MODE', 'dev').lower()
311             is_debug = os.getenv('DEBUG', 'true').lower()
312             == 'true'
313             self.is_debug_mode = mode_val in ('dev', 'debu
g') or is_debug
314         except Exception:
315             self.is_debug_mode = True
316
317     def _setup_file_handlers(self) -> None:
318         """Sets обработчики для файловых логов."""
319         self.logger_file_info: logging.Logger = logging.getLog
320         ger('logger_file_info')
321         self.logger_file_info.setLevel(logging.INFO)
322         self.logger_file_info.propagate = False
323         info_handler = logging.FileHandler(self.info_log_path,
324         encoding='utf-8')
325         info_handler.setFormatter(logging.Formatter('%(levelna
326         me)s: %(message)s'))
327         self.logger_file_info.addHandler(info_handler)
328         self.logger_file_debug: logging.Logger = logging.getLo
329         gger('logger_file_debug')
330         self.logger_file_debug.setLevel(logging.DEBUG)
331         self.logger_file_debug.propagate = False
332         debug_handler = logging.FileHandler(self.debug_log_pat
333         h, encoding='utf-8')
334         debug_handler.setFormatter(logging.Formatter('%(leveln
335         ame)s: %(message)s'))
336         self.logger_file_debug.addHandler(debug_handler)
337         self.logger_file_errors: logging.Logger = logging.getL
338         ogger('logger_file_errors')
339         self.logger_file_errors.setLevel(logging.ERROR)
340         self.logger_file_errors.propagate = False
341         errors_handler = CompressingHandler(str(self.errors_lo
342         g_path), encoding='utf-8')

```

```

334 |         errors_handler.setFormatter(logging.Formatter('%(level
| name)s: %(message)s'))
335 |         self.logger_file_errors.addHandler(errors_handler)
336 |         self.logger_file_json: logging.Logger = logging.getLog
| ger('logger_json')
337 |         self.logger_file_json.setLevel(logging.DEBUG)
338 |         self.logger_file_json.propagate = False
339 |         json_handler = logging.FileHandler(self.json_log_path,
| encoding='utf-8')
340 |         json_handler.setFormatter(JsonFormatter())
341 |         self.logger_file_json.addHandler(json_handler)
342 |
343 |     def _setup_module_loggers(self) -> None:
344 |         """Sets логгеры для отдельных модулей."""
345 |         formatter = logging.Formatter('%(asctime)s - %(levelna
| me)s - %(message)s')
346 |         modules_config = [('logger_fastapi', self.fastapi_log_
| path), ('logger_gemini', self.gemini_log_path), ('logger_playw
| right', self.playwright_log_path), ('logger_yt_dlp', self.yt_d
| lp_log_path)]
347 |         for logger_name, log_path in modules_config:
348 |             module_logger = logging.getLogger(logger_name)
349 |             module_logger.setLevel(logging.DEBUG)
350 |             module_logger.propagate = False
351 |             handler = CompressingHandler(str(log_path), encodi
| ng='utf-8')
352 |             handler.setFormatter(formatter)
353 |             module_logger.addHandler(handler)
354 |             setattr(self, logger_name, module_logger)
355 |
356 |     def _get_caller_info(self, depth: int=2) -> Tuple[str, str
| , int]:
357 |         """
358 |         Безопасно receives информацию о вызывающей функции.
359 |
360 |         Args:
361 |             depth: Глубина стека для поиска caller'a.
362 |

```

```

363     Returns:
364         Tuple (file_name, function_name, line_number).
365     """
366     try:
367         stack = inspect.stack()
368         if len(stack) > depth:
369             frame_info = stack[depth]
370             return (frame_info.filename, frame_info.function, frame_info.lineno)
371     except Exception:
372         pass
373     return ('unknown', 'unknown', 0)
374
375     def _format_message(self, message: Any, ex: Optional[Exception]=None, color: Optional[Tuple[str, str]]=None) -> str:
376         """
377         Форматирует сообщение с опциональным цветом и информацией об исключении.
378
379         Args:
380             message: Сообщение или объект для форматирования.
381             ex: Exception для добавления.
382             color: Tuple (текст_цвет, фон_цвет).
383
384         Returns:
385             Отформатированное сообщение.
386         """
387         from src.utils.printer import pformat
388         formatted = pformat(message)
389         if ex:
390             formatted = f'{formatted} {str(ex)}'
391         if color:
392             text_color, bg_color = color
393             text_color = TEXT_COLORS.get(text_color, colorama.Fore.RESET)
394             bg_color = BG_COLORS.get(bg_color, colorama.Back.RESET)
395         return f'{text_color}{bg_color}{formatted}{colorama

```

```

a.Style.RESET_ALL}'
396         return formatted
397
398     def _route_to_module_logger(self, message: str, level: int
, filename: str) -> None:
399         """
400         Маршрутизирует лог в Module-специфичный логгер на осно
ве пути файла.
401
402         Args:
403             message: Текст сообщения.
404             level: Уровень логирования.
405             filename: Путь к файлу вызывающего кода.
406         """
407         filename_lower = filename.lower().replace('\\', '/')
408         routing_map = [('fastapi', self.logger_fastapi), ('mai
n.py', self.logger_fastapi), ('gemini', self.logger_gemini), (
'src/ai', self.logger_gemini), ('src\\ai', self.logger_gemini)
, ('playwright', self.logger_playwright), ('torrent_playwright
', self.logger_playwright), ('yt_dlp', self.logger_yt_dlp), ('
yt-dlp', self.logger_yt_dlp)]
409         for pattern, module_logger in routing_map:
410             if pattern.lower() in filename_lower:
411                 try:
412                     module_logger.log(level, message)
413                 except Exception:
414                     pass
415                 break
416
417     def setLevel(self, level: int | str) -> None:
418         """Устанавливает уровень логирования для совместимости
.
419
420         Args:
421             level: Уровень логирования (число или имя уровня).
422         """
423         if isinstance(level, str):
424             self.is_debug_mode = level.upper() == 'DEBUG'

```

```

425         elif isinstance(level, int):
426             self.is_debug_mode = level <= logging.DEBUG
427
428     def log(self, level: int, message: Any, ex: Optional[Exception]=None, exc_info: bool=False, color: Optional[Tuple[str, str]]=None) -> None:
429         """
430         Логирует сообщение с заданным уровнем и параметрами.
431
432         Args:
433             level: Уровень логирования (logging.INFO, logging.
434             ERROR и т.д.).
435             message: Текст сообщения или структурированные дан
436             ные.
437             ex: Exception для логирования.
438             exc_info: Включить полную информацию об исключении
439             .
440             color: Tuple (текст_цвет, фон_цвет) для окраски.
441         """
442         if level == logging.DEBUG and (not self.is_debug_mode)
443         :
444             return
445         from src.utils.printer import pformat
446         plain_formatted = pformat(message)
447         if ex:
448             plain_formatted = f'{plain_formatted} {str(ex)}'
449         colored_message = self._format_message(message, ex, co
450         lor)
451         if self.logger_console:
452             try:
453                 self.logger_console.log(level, colored_message
454                 , exc_info=exc_info)
455             except Exception:
456                 pass
457         if self.logger_file_json:
458             self.logger_file_json.log(level, plain_formatted,
459             exc_info=exc_info)
460         if level == logging.INFO and self.logger_file_info:

```

```

454         self.logger_file_info.log(level, plain_formatted)
455     elif level == logging.DEBUG and self.logger_file_debug
456     :
457         self.logger_file_debug.log(level, plain_formatted)
458     elif level in [logging.ERROR, logging.CRITICAL] and se
459     lf.logger_file_errors:
460         self.logger_file_errors.log(level, plain_formatted
461         )
462     try:
463         filename, func_name, line_no = self._get_caller_in
464         fo(depth=3)
465         self._route_to_module_logger(plain_formatted, leve
466         l, filename)
467     except Exception:
468         pass
469
470     def _cleanup(self) -> None:
471         """Очищает ресурсы при завершении приложения."""
472         try:
473             for logger_name in ['logger_console', 'logger_file
474             _info', 'logger_file_debug', 'logger_file_errors', 'logger_jso
475             n', 'logger_fastapi', 'logger_gemini', 'logger_playwright', 'l
476             ogger_yt_dlp']:
477                 logger_obj = getattr(self, logger_name, None)
478                 if logger_obj:
479                     for handler in logger_obj.handlers[:]:
480                         handler.flush()
481                         handler.close()
482                         logger_obj.removeHandler(handler)
483         except Exception:
484             pass
485
486     def info(self, message: Any, ex: Optional[Exception]=None,
487             exc_info: bool=False, text_color: str='green', bg_color: str=
488             '') -> None:
489         """Логирует сообщение уровня INFO с зелёным цветом по
490         умолчанию."""
491         color = (text_color, bg_color) if bg_color else (text_

```

```
color, '')
481         self.log(logging.INFO, message, ex, exc_info, color)
482
483     def success(self, message: Any, ex: Optional[Exception]=None, exc_info: bool=False, text_color: str='light_green', bg_color: str='') -> None:
484         """Логирует сообщение об успешной операции с жёлтым цветом по умолчанию."""
485         color = (text_color, bg_color) if bg_color else (text_color, '')
486         self.log(logging.INFO, message, ex, exc_info, color)
487
488     def warning(self, message: Any, ex: Optional[Exception]=None, exc_info: bool=False, text_color: str='black', bg_color: str='yellow') -> None:
489         """Логирует сообщение уровня WARNING с чёрным текстом на жёлтом фоне."""
490         color = (text_color, bg_color)
491         self.log(logging.WARNING, message, ex, exc_info, color)
492
493     def debug(self, message: Any, ex: Optional[Exception]=None, exc_info: bool=False, text_color: str='cyan', bg_color: str='') -> None:
494         """Логирует сообщение уровня DEBUG с голубым цветом."""
495         color = (text_color, bg_color) if bg_color else (text_color, '')
496         self.log(logging.DEBUG, message, ex, exc_info, color)
497
498     def error(self, message: Any, ex: Optional[Exception]=None, exc_info: bool=True, text_color: str='red', bg_color: str='') -> None:
499         """Логирует сообщение уровня ERROR с красным цветом и информацией об исключении."""
500         color = (text_color, bg_color) if bg_color else (text_color, '')
501         self.log(logging.ERROR, message, ex, exc_info, color)
```

```
502 |
503 |     def critical(self, message: Any, ex: Optional[Exception]=None, exc_info: bool=True, text_color: str='white', bg_color: str='red') -> None:
504 |         """Логирует критическую ошибку с белым текстом на красном фоне."""
505 |         color = (text_color, bg_color)
506 |         self.log(logging.CRITICAL, message, ex, exc_info, color)
507 | logger: Logger = Logger()
508 | 'Глобальный экземпляр логгера для использования во всем приложении.'
```

log_analyzer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Logger - Log Analyzer
4 # =====
5 # Description:
6 #   Intelligent log analyzer with Gemini AI integration.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from logger.log_analyzer import get_max_size_bytes
11 #
12 #       res = get_max_size_bytes()
13 #
14 # File: log_analyzer.py
15 # Project: ai-breadboard
16 # Package: logger
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:29:02
20 # =====
21
22 """Intelligent log analyzer with Gemini AI integration.
23
24 Provides functionality for analyzing application logs using Go
25 oogle Generative AI (Gemini),
26 automatic report generation, and Master Journal maintenance fo
27 r system state tracking.
28
29 Functions:
30     - Log analysis with AI assistance
31     - Automatic report generation
32     - Master Journal creation and maintenance
33     - Detailed error report creation
```

```

32 |     - System state tracking
33 |     - Intelligent log rotation and cleanup"""
34 |
35 | import os
36 | import asyncio
37 | import datetime
38 | import logging
39 | import tempfile
40 | from pathlib import Path
41 | from typing import Optional, List
42 | from concurrent.futures import ThreadPoolExecutor
43 | from src.ai import UnifiedChatModel
44 | from logger import logger
45 | from header import __root__
46 | LOG_DIR: Path = Path(tempfile.gettempdir()) / 'ai-breadboard'
47 | / 'logs'
48 | REPORTS_DIR: Path = Path(tempfile.gettempdir()) / 'ai-breadboa
49 | rd' / 'reports'
50 | LOCK_FILE: Path = LOG_DIR / 'log_analyzer.lock'
51 | DEFAULT_MAX_SIZE_MB: float = 10.0
52 | MAX_CHARS_FOR_ANALYSIS: int = 500 * 1024
53 | CHECK_INTERVAL_SECONDS: int = 60
54 | MAX_REPORTS_FOR_JOURNAL: int = 10
55 |
56 | def get_max_size_bytes() -> float:
57 |     """
58 |     Receives максимальный размер лог-файла из конфигурации.
59 |
60 |     Returns:
61 |         float: Максимальный размер в байтах.
62 |
63 |     Raises:
64 |         Returns default value (10MB) if config is not availabl
65 | e.
66 |     """
67 |     try:
68 |         from src.config import logging_cfg
69 |         mb = float(getattr(logging_cfg, 'max_size_mb', DEFAULT

```

```

    _MAX_SIZE_MB)) if logging_cfg else DEFAULT_MAX_SIZE_MB
67     return mb * 1024 * 1024
68     except Exception as ex:
69         logger.debug(f'Не удалось прочитать конфиг размера лог
a: {ex}')
70         return DEFAULT_MAX_SIZE_MB * 1024 * 1024
71
72 async def analyze_log_file(file_path: Path, ai_model: UnifiedC
hatModel) -> bool:
73     """
74     Анализирует лог-файл с помощью AI и creates отчёт.
75
76     Процесс:
77     1. Reads содержимое лог-файла
78     2. Sends на анализ Gemini AI (максимум 500KB для экономии
квоты)
79     3. Saves результат в виде Markdown отчёта
80     4. Очищает исходный лог-файл
81
82     Args:
83         file_path: Путь к лог-файлу для анализа.
84         ai_model: Инициализированный экземпляр UnifiedChatMode
l.
85
86     Returns:
87         bool: True если анализ прошёл successfully, False в сл
учае ошибки.
88     """
89     try:
90         if not file_path.exists() or not file_path.is_file():
91             logger.warning(f'Лог-файл не найден: {file_path}')
92             return False
93         try:
94             with open(file_path, 'r', encoding='utf-8', errors
='ignore') as f:
95                 content = f.read()
96         except Exception as ex:
97             logger.error(f'Error при чтении лог-файла {file_pa

```

```

th.name}: {ex}')
98         return False
99         if not content.strip():
100             logger.debug(f'Лог-файл {file_path.name} пуст, про
пуск анализа')
101             return True
102             logger.info(f'Начало умного анализа лога {file_path.name} (размер: {len(content)} символов)...')
103             if len(content) > MAX_CHARS_FOR_ANALYSIS:
104                 logger.warning(f'Лог {file_path.name} слишком больш
шой ({len(content)} символов), анализирую только последние {MAX_CHARS_FOR_ANALYSIS} символов')
105                 content = content[-MAX_CHARS_FOR_ANALYSIS:]
106                 prompt = f'\nПроанализируй следующий лог-файл ({file_path.name}).\nВыяви ошибки, предупреждения, критические проблем
ы, а также общие тренды и дай рекомендации по исправлению.\nОт
вет предоставь на русском языке в чистом формате Markdown.\n\n
Содержимое лога:\n{content}\n'
107                 try:
108                     report_text = await ai_model.ask(prompt)
109                 except Exception as ex:
110                     logger.error(f'Error при запросе к Gemini AI: {ex}
')
111                     report_text = f'# Отчет об анализе лога {file_path.name}\n\nError при запросе к AI: {ex}'
112                     if not report_text:
113                         report_text = f'# Отчет об анализе лога {file_path.name}\n\nНе удалось получить анализ от модели Gemini.'
114                     REPORTS_DIR.mkdir(parents=True, exist_ok=True)
115                     timestamp = datetime.datetime.now().strftime('%Y%m%d_%H%M%S')
116                     report_name = f'report_{file_path.stem}_{timestamp}.md'
117                     report_path = REPORTS_DIR / report_name
118                     try:
119                         with open(report_path, 'w', encoding='utf-8') as f
:
120                             f.write(report_text)

```

```

121         except Exception as ex:
122             logger.error(f'Error при сохранении отчёта {report
_name}: {ex}')
123             return False
124         try:
125             with open(file_path, 'w', encoding='utf-8') as f:
126                 f.truncate(0)
127         except Exception as ex:
128             logger.error(f'Error при очистке лог-файла {file_p
ath.name}: {ex}')
129             logger.info(f'␣ Лог {file_path.name} successfully проа
нализован. Отчет сохранен в {report_name}')
130             return True
131         except Exception as ex:
132             logger.error(f'Критическая Error при анализе лог-файла
{file_path.name}: {ex}')
133             return False
134
135 async def update_master_journal(ai_model: UnifiedChatModel) ->
bool:
136     """
137     Creates главный журнал (Master Journal) из последних отчёт
ов анализа.
138
139     Процесс:
140     1. Собирает последние N отчётов об анализе
141     2. Sends их на анализ Gemini для синтеза
142     3. Saves результат в master_journal.md
143
144     Args:
145         ai_model: Инициализированный экземпляр UnifiedChatMode
l.
146
147     Returns:
148         bool: True если журнал successfully обновлён, False в
случае ошибки.
149     """
150     try:

```

```
151         if not REPORTS_DIR.exists():
152             logger.debug('Директория отчётов не существует, пр
опуск обновления журнала')
153             return False
154         report_files = [p for p in REPORTS_DIR.glob('report_*.
md') if p.name != 'master_journal.md']
155         if not report_files:
156             logger.debug('Нет отчётов для создания главного жу
рнала')
157             return False
158         report_files.sort(key=lambda p: p.stat().st_mtime, rev
erse=True)
159         reports_content: List[str] = []
160         for p in report_files[:MAX_REPORTS_FOR_JOURNAL]:
161             try:
162                 with open(p, 'r', encoding='utf-8', errors='ig
nore') as f:
163                     content = f.read()
164                     reports_content.append(f'### Файл: {p.name
}\n{content}\n')
165             except Exception as ex:
166                 logger.warning(f'Error при чтении отчёта {p.na
me}: {ex}')
167                 continue
168         if not reports_content:
169             logger.warning('Не удалось прочитать ни один отчёт
для главного журнала')
170             return False
171         logger.info(f'Создание главного журнала из {len(report
s_content)} отчётов...')
172         prompt = f"\nНа основе следующих индивидуальных отчето
в об анализе логов составь общий единый журнал (Master Journal
) состояния системы.\nВыдели:\n1. Повторяющиеся и постоянные о
шибки.\n2. Новые обнаруженные тенденции.\n3. Актуальные пробле
мы, требующие внимания разработчика на текущий момент.\n4. Уст
аревшие или неактуальные проблемы, которые перестали появлятьс
я.\n5. Рекомендации по приоритизации исправлений.\n\n0ответ выв
еди строго на русском языке в формате Markdown с чётким структ
```

```

урированием информации.\n\nИндивидуальные отчеты:\n{''.join(re
ports_content))\n"
173         try:
174             master_text = await ai_model.ask(prompt)
175         except Exception as ex:
176             logger.error(f'Error при запросе к Gemini AI для г
лавного журнала: {ex}')
177             master_text = f'# Общий журнал анализа логов\n\nEr
ror при создании синтеза: {ex}'
178             if not master_text:
179                 master_text = '# Общий журнал анализа логов\n\nНе
удалось получить общий анализ от модели Gemini.'
180             master_path = REPORTS_DIR / 'master_journal.md'
181             try:
182                 with open(master_path, 'w', encoding='utf-8') as f
:
183                     f.write(master_text)
184             except Exception as ex:
185                 logger.error(f'Error при сохранении главного журна
ла: {ex}')
186                 return False
187             logger.info('□ Общий журнал (master_journal.md) succes
sfully обновлен')
188             return True
189         except Exception as ex:
190             logger.error(f'Критическая Error при обновлении общего
журнала логов: {ex}')
191             return False
192
193 async def log_analyzer_loop() -> None:
194     """
195     Главный цикл анализатора логов, выполняемый в фоновом пото
ке.
196
197     Функциональность:
198     1. Checks конфигурацию включения анализатора
199     2. Инициализирует AI модель
200     3. Периодически (каждые N секунд) checks размер лог-файлов

```

```
201 |     4. Запускает анализ файлов превысивших лимит размера
202 |     5. Обновляет главный журнал после каждого анализа
203 |
204 |     Runs indefinitely until application shutdown.
205 |     """
206 |     logger.info('Запуск фоновой службы интеллектуального анали
за логов...')
207 |     try:
208 |         from src.config import logging_cfg
209 |         if not logging_cfg or not getattr(logging_cfg, 'enable
_log_analyzer', False):
210 |             logger.warning('Анализ логов отключён в config.jso
n')
211 |             return
212 |     except Exception as ex:
213 |         logger.warning(f'Не удалось прочитать конфиг анализато
ра: {ex}')
214 |         return
215 |     try:
216 |         api_key_names = [n.strip() for n in os.getenv('GEMINI_
API_KEY_NAMES', '').split(',') if n.strip()]
217 |         if not api_key_names:
218 |             logger.warning('Переменная окружения GEMINI_API_KE
Y_NAMES не установлена, анализ логов отключен')
219 |             return
220 |         system_instruction = 'Вы – профессиональный аналитик с
истемных логов. Ваша задача – исследовать логи, выявлять ошибк
и, проблемы, тренды и давать рекомендации по устранению.'
221 |         ai_model = UnifiedChatModel(system_instruction=system_
instruction)
222 |     except Exception as ex:
223 |         logger.error(f'Error при инициализации AI модели: {ex}
')
224 |         return
225 |     logger.info('Анализатор логов успешно инициализирован')
226 |     while True:
227 |         try:
228 |             if not LOG_DIR.exists():
```

```

229         await asyncio.sleep(CHECK_INTERVAL_SECONDS)
230         continue
231         max_bytes = get_max_size_bytes()
232         for log_file in LOG_DIR.glob('*.log'):
233             if log_file.is_file() and log_file.stat().st_s
size >= max_bytes:
234                 success = await analyze_log_file(log_file,
ai_model)
235                 if success:
236                     await update_master_journal(ai_model)
237                     await asyncio.sleep(1)
238                 json_log = LOG_DIR / 'log.json'
239                 if json_log.exists() and json_log.is_file() and (j
son_log.stat().st_size >= max_bytes):
240                     success = await analyze_log_file(json_log, ai_
model)
241                     if success:
242                         await update_master_journal(ai_model)
243             except Exception as ex:
244                 logger.error(f'Error в цикле анализатора логов: {e
x}')
245                 await asyncio.sleep(CHECK_INTERVAL_SECONDS)
246
247 def start_log_analyzer() -> None:
248     """
249     Запускает анализатор логов в фоновом режиме.
250
251     Особенности:
252     - Гарантирует, что анализатор запускается только в одном п
роцессе
253     - Использует lock-файл для синхронизации между процессами
254     - Checks живость процесса перед запуском нового
255     - Безопасно processes ошибки инициализации
256
257     Raises:
258     Не выбрасывает исключений, все ошибки логируются.
259     """
260     try:

```

```

261 |         LOG_DIR.mkdir(parents=True, exist_ok=True)
262 |         if LOCK_FILE.exists():
263 |             try:
264 |                 with open(LOCK_FILE, 'r', encoding='utf-8') as
f:
265 |                     pid = int(f.read().strip())
266 |                     try:
267 |                         import psutil
268 |                         if psutil.pid_exists(pid):
269 |                             logger.warning(f'Анализатор логов уже
запущен (PID: {pid})')
270 |                             return
271 |                     except ImportError:
272 |                         logger.debug('psutil не установлен, пропус
к проверки процесса')
273 |                         return
274 |                     except Exception as ex:
275 |                         logger.debug(f'Не удалось проверить процес
с {pid}: {ex}')
276 |                 except (ValueError, OSError) as ex:
277 |                     logger.debug(f'Error при чтении lock-файла: {e
x}')
278 |                 try:
279 |                     LOCK_FILE.unlink(missing_ok=True)
280 |                 except Exception as ex:
281 |                     logger.warning(f'Не удалось удалить старый loc
k-файл: {ex}')
282 |                 try:
283 |                     import subprocess
284 |                     pid = subprocess.os.getpid()
285 |                     LOCK_FILE.write_text(str(pid), encoding='utf-8')
286 |                 except Exception as ex:
287 |                     logger.error(f'Error при создании lock-файла: {ex}
')
288 |                 return
289 |             try:
290 |                 task = asyncio.create_task(log_analyzer_loop())
291 |                 logger.debug('Асинхронная задача анализатора логов

```

```
    создана')
292 |         except RuntimeError as ex:
293 |             logger.debug(f'Event loop не активен ({ex}), пропу
    ск запуска анализатора')
294 |         except Exception as ex:
295 |             logger.error(f'Error при запуске анализатора логов: {e
    x}')
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Logger - Init
4 # =====
5 # Description:
6 #   Скрипт/модуль системы AI-Breadboard (`__init__`).
7 #
8 # Usage Examples:
9 #
10 #
11 # File: __init__.py
12 # Project: ai-breadboard
13 # Package: logger
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # Updated: 2026-10-01 13:29:02
17 # =====
18
19 """Скрипт/модуль системы AI-Breadboard (`__init__`)."""
20
21 from .logger import Logger, logger, JsonFormatter
```